

17.11.2025

Хэш-функции и хеш-таблицы. Указатель на функцию.

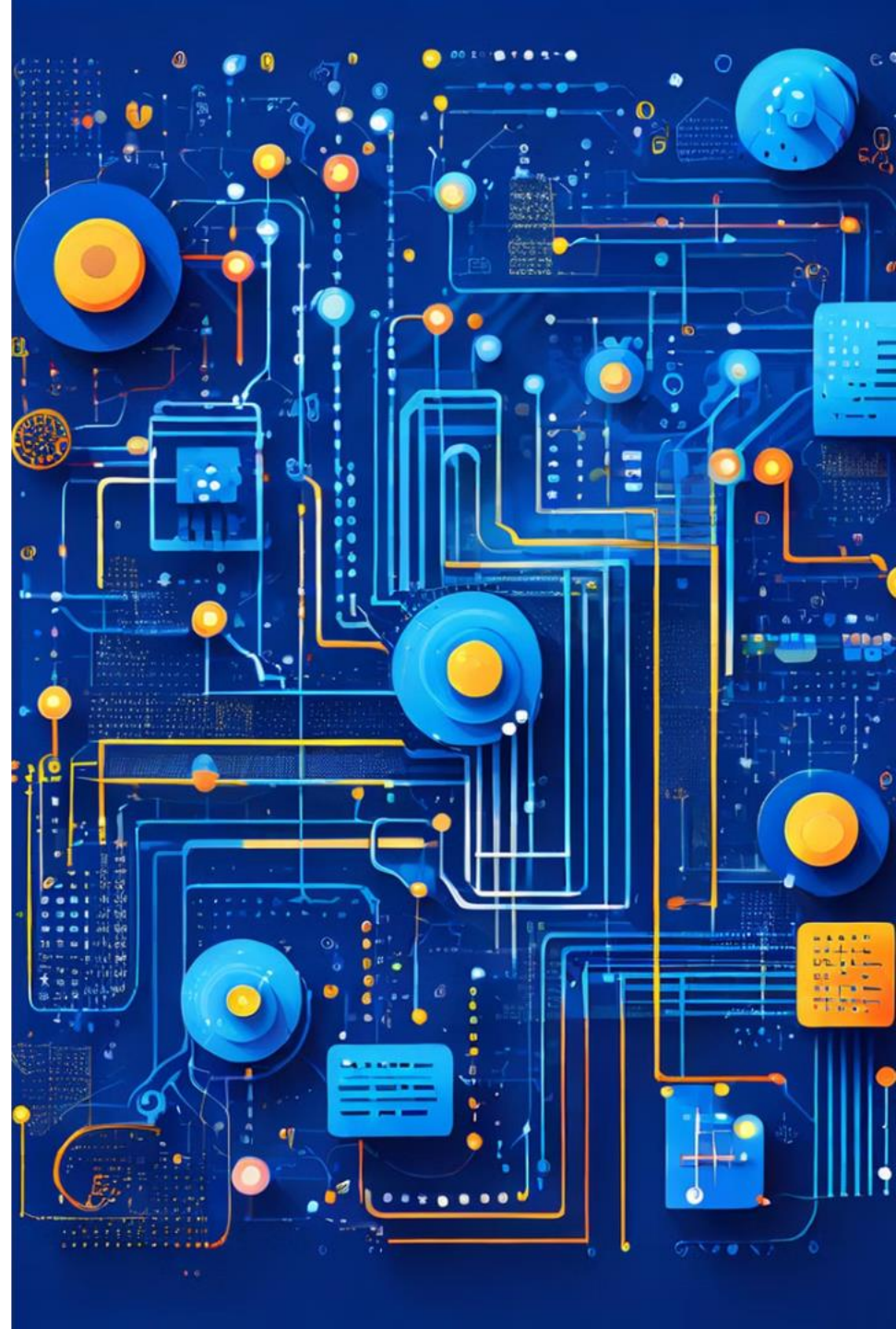
Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

N * Новосибирский
государственный
университет
***НАСТОЯЩАЯ НАУКА**

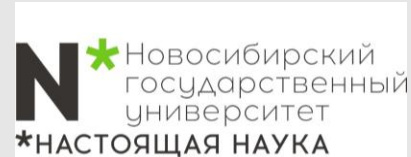


Давайте познакомимся



Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.



Agenda

**Хеш-функции и
хеш-таблицы**

85 минут

**Указатель на
функцию**

5 минут

Agenda

**Хеш-функции и
хеш-таблицы**

85 минут

**Указатель на
функцию**

5 минут

Введение

Мы знаем два алгоритма поиска: линейный поиск и бинарный поиск. Линейный поиск имеет время выполнения, пропорциональное $O(n)$, в то время как бинарный поиск занимает время, пропорциональное $O(\log n)$, где n — количество элементов в массиве. Двоичный поиск и деревья бинарного поиска являются эффективными алгоритмами для поиска элемента. Но что, если мы хотим выполнить операцию поиска за время, пропорциональное $O(1)$? Другими словами, есть ли способ выполнить поиск в массиве за постоянное время, независимо от его размера?

Ключ	Массив записей сотрудников
Ключ 0 -> [0]	Запись сотрудника с Emp_ID 0
Ключ 1 -> [1]	Запись сотрудника с Emp_ID 1
Ключ 2 -> [2]	Запись сотрудника с Emp_ID 2
.....	
.....	
Ключ 98 -> [98]	Запись сотрудника с Emp_ID 98
Ключ 99 -> [99]	Запись сотрудника с Emp_ID 99



Введение

Предположим, что та же компания использует пятизначный Emp_ID в качестве первичного ключа. В этом случае значения ключей будут находиться в диапазоне от 00000 до 99999. Если мы хотим использовать ту же технику, что и выше, нам понадобится массив размером 100 000, из которых будут использоваться только 100 элементов.

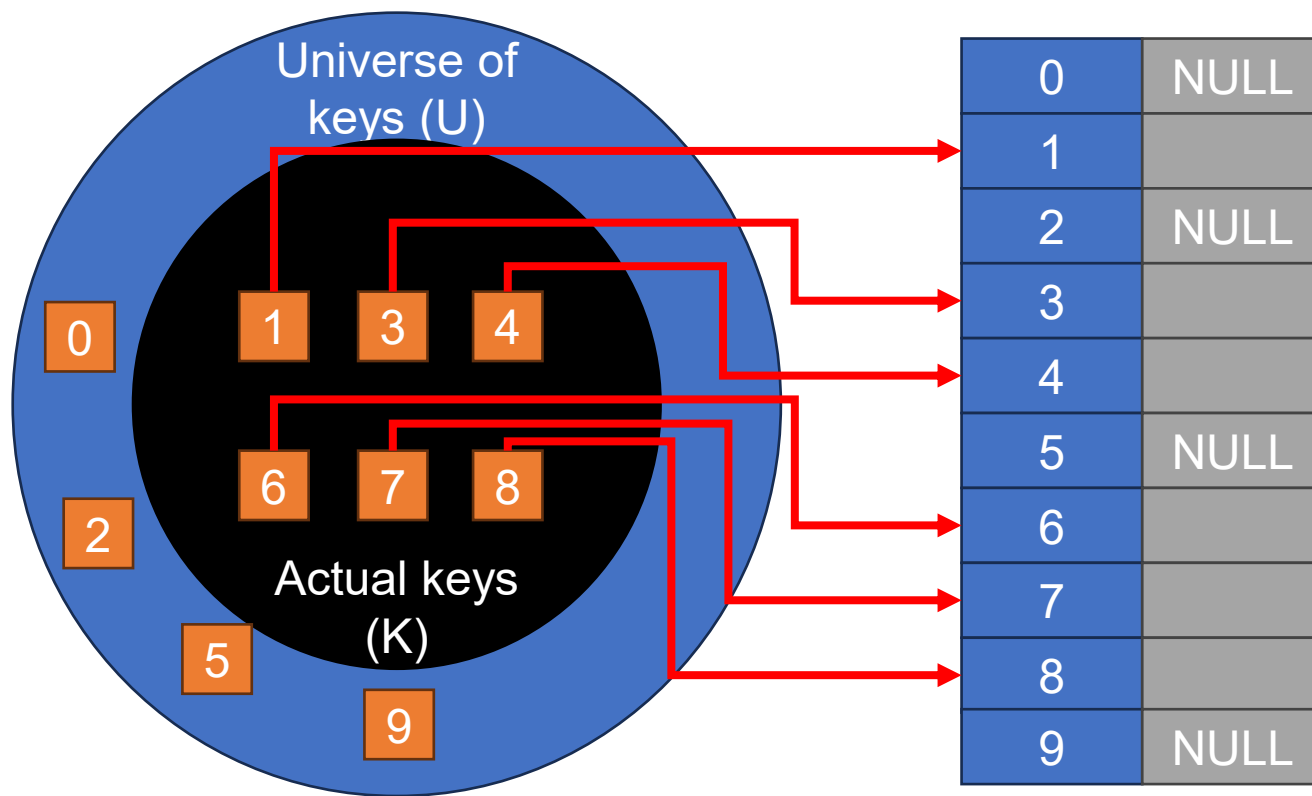
Ключ	Массив записей сотрудников
Ключ 00000 -> [0]	Запись сотрудника с Emp_ID 0
.....	
Ключ n -> [n]	Запись сотрудника с Emp_ID n
.....	
Ключ 99998 -> [99998]	Запись сотрудника с Emp_ID 98998
Ключ 99999 -> [99999]	Запись сотрудника с Emp_ID 99999

Нецелесообразно тратить так много места для хранения только для того, чтобы гарантировать, что запись каждого сотрудника находится в уникальном и предсказуемом месте.



Хэш-таблицы

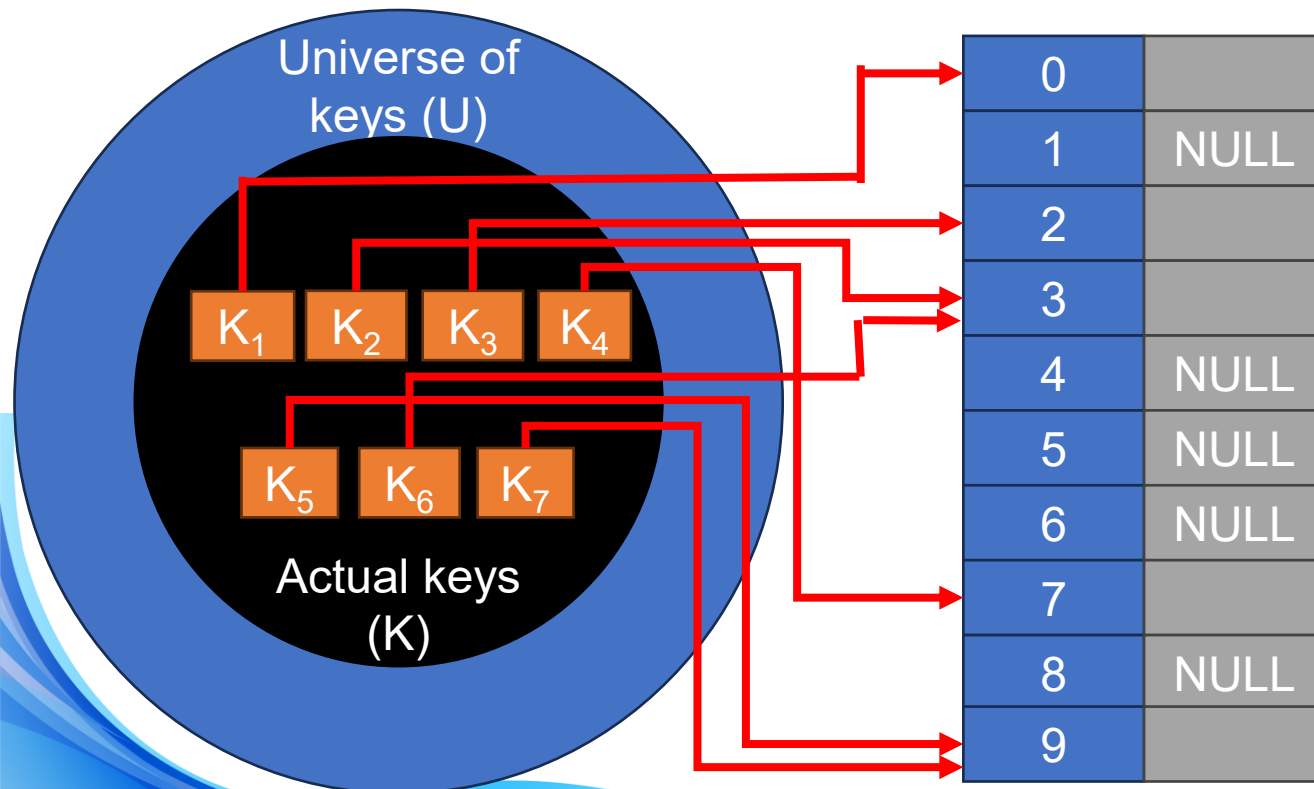
Хэш-таблица — это структура данных, в которой ключи сопоставляются с позициями массива с помощью хэш-функции. В обсуждаемом здесь примере мы будем использовать хэш-функцию, которая извлекает последние две цифры ключа. Поэтому мы сопоставляем ключи с позициями массива или индексами массива. Значение, хранящееся в хэш-таблице, можно найти за время $O(1)$ с помощью хэш-функции, которая генерирует адрес из ключа (создавая индекс массива, в котором хранится значение).



Эта концепция полезна, когда общая вселенная ключей невелика и когда большинство ключей фактически используются из всего набора ключей. Это эквивалентно нашему первому примеру, где есть 100 ключей для 100 сотрудников. Однако, когда набор K фактически используемых ключей меньше, чем вселенная ключей (U), хэш-таблица потребляет меньше места для хранения. Требование к хранению для хэш-таблицы составляет $O(k)$, где k — это количество фактически используемых ключей.

Хэш-таблицы

В хэш-таблице элемент с ключом k хранится в индексе $h(k)$, а не k . Это означает, что хэш-функция h используется для вычисления индекса, в котором будет храниться элемент с ключом k . Этот процесс сопоставления ключей с соответствующими позициями (или индексами) в хэш-таблице называется хэшированием.



Обратите внимание, что ключи K_2 и K_6 указывают на одну и ту же ячейку памяти. Это известно как коллизия. То есть, когда два или более ключей сопоставлены с одной и той же ячейкой памяти, говорят, что происходит коллизия. Аналогично, ключи K_5 и K_7 также сталкиваются. Основная цель использования хэш-функции — сократить диапазон индексов массива, которые необходимо обработать. Таким образом, вместо значений U нам нужны только значения K , тем самым уменьшая объем требуемого пространства для хранения.

Хеш-функции

Хеш-функция — это математическая формула, которая при применении к ключу создает целое число, которое можно использовать в качестве индекса для ключа в хеш-таблице. Основная цель хеш-функции заключается в том, чтобы элементы были распределены относительно, случайно и равномерно. Она создает уникальный набор целых чисел в некотором подходящем диапазоне, чтобы уменьшить количество коллизий. На практике не существует хеш-функции, которая полностью устраняет коллизии. Хорошая хеш-функция может только минимизировать количество коллизий, равномерно распределяя элементы по всему массиву.



Свойства хорошей хеш-функции

Низкая стоимость Стоимость выполнения хеш-функции должна быть небольшой, чтобы использование метода хеширования стало предпочтительным по сравнению с другими подходами. Например, если алгоритм бинарного поиска может выполнять поиск элемента в отсортированной таблице из n элементов с помощью $\log_2 n$ сравнений ключей, то хеш-функция должна обходиться дешевле, чем выполнение $\log_2 n$ сравнений ключей.

Детерминизм Процедура хеширования должна быть детерминированной. Это означает, что для заданного входного значения должно быть сгенерировано одно и то же значение хеширования. Однако этот критерий исключает функции хеширования, которые зависят от внешних переменных параметров (таких как время суток) и от адреса памяти хешируемого объекта (поскольку адрес объекта может измениться во время обработки).

Равномерность Хорошая функция хеширования должна отображать ключи как можно более равномерно по всему диапазону выходных данных. Это означает, что вероятность генерации каждого значения хеширования в диапазоне выходных данных должна быть примерно одинаковой. Свойство равномерности также минимизирует количество коллизий.

Различные функции хеширования

- метод деления

Это самый простой метод хеширования целого числа x . Этот метод делит x на M , а затем использует полученный остаток. В этом случае хэш-функция может быть задана как

$$h(x) = x \bmod M$$

Метод деления довольно хорош практически для любого значения M , и поскольку он требует только одной операции деления, метод работает очень быстро. Однако следует проявить особую осторожность, чтобы выбрать подходящее значение для M .

Пример кода

```
int const M = 97; // простое число
int h (int x) {
    return (x % M);
}
```

Пример: хеш-значения ключей 1234 и 5462.

Решение

При $M = 97$ хеш-значения можно вычислить следующим образом:

$$h(1234) = 1234 \% 97 = 70$$

$$h(5642) = 5642 \% 97 = 16$$

Интересная статья про [ускорение взятия остатка](#)

Различные функции хеширования

- метод умножения

Шаги, используемые в методе умножения, следующие:

Шаг 1: Выберите константу A , такую, что $0 < A < 1$.

Шаг 2: Умножьте ключ k на A .

Шаг 3: Извлеките дробную часть kA .

Шаг 4: Умножьте результат шага 3 на размер хеш-таблицы (m).

Следовательно, хэш-функция может быть задана как:

$$h(k) = m (kA \bmod 1)$$

Кнут предположил, что лучшим выбором A является $(\sqrt{5} - 1) / 2 = 0,6180339887$

пример

Учитывая хеш-таблицу размером 1000, сопоставьте ключ 12345 с соответствующим местом в хеш-таблице.

Решение

Мы будем использовать:

$A = 0,618033$, $m = 1000$ и $k = 12345$

$h(12345) = 1000 (12345 \times 0,618033 \bmod 1)$

$h(12345) = 1000 (7629,617385 \bmod 1)$

$h(12345) = 1000 (0,617385)$

$h(12345) = 617,385$

$h(12345) = 617$

Различные функции хеширования

- метод середины квадрата

Метод середины квадрата — это хорошая хеш-функция, которая работает в два этапа:

Шаг 1: Возвести в квадрат значение ключа. То есть найти k^2 .

Шаг 2: Извлечь средние r цифр результата, полученного на Шаге 1.

Алгоритм работает хорошо, потому что большинство или все цифры значения ключа вносят вклад в результат. Хеш-функция может быть задана как:

$$h(k) = s$$

где s получается путем выбора r цифр из k^2 .

пример

Рассчитайте хеш-значение для ключей 1234 и 5642 с помощью метода середины квадрата. Хеш-таблица имеет 100 ячеек памяти.

Решение

Обратите внимание, что хеш-таблица имеет 100 ячеек памяти, индексы которых варьируются от 0 до 99. Это означает, что для сопоставления ключа с ячейкой в хеш-таблице требуется всего две цифры, поэтому $r = 2$.

Когда $k = 1234$, $k^2 = 1522756$, $h(1234) = 27$

Когда $k = 5642$, $k^2 = 31832164$, $h(5642) = 21$

Обратите внимание, что выбраны 3-я и 4-я цифры, начиная справа.

Различные функции хеширования

- метод сворачивания

Метод сворачивания работает в следующие два шага:

Шаг 1: Разделите значение ключа на несколько частей. То есть, разделите k на части $k_1, k_2, \dots, k_n$, где каждая часть имеет одинаковое количество цифр, за исключением последней части, которая может иметь меньше цифр, чем другие части.

Шаг 2: Сложите отдельные части. То есть, получите сумму $k_1 + k_2 + \dots + k_n$. Значение хэша получается путем игнорирования последнего переноса, если таковой имеется.

пример

Учитывая хэш-таблицу из 100 ячеек, вычислите хэш-значение с помощью метода сворачивания для ключей 5678, 321 и 34567.

Решение

Поскольку необходимо адресовать 100 ячеек памяти, мы разобьем ключ на части, каждая из которых (кроме последней) будет содержать две цифры. Значения хэш-функции можно получить следующим образом:

Ключ	5678	321	34567
Части	56 и 78	32 и 1	34, 56 и 7
Сумма	134	33	97
Хэш-значение	34 (134 % 100)	33	97

Коллизии

Коллизии возникают, когда хэш-функция сопоставляет два разных ключа с одним и тем же местом. Очевидно, что две записи не могут храниться в одном и том же месте. Поэтому применяется метод, используемый для решения проблемы коллизий, также называемый методом разрешения коллизий.

Два самых популярных метода разрешения коллизий:

1. Открытая адресация
2. Цепочка



Разрешение коллизий с помощью открытой адресации

После возникновения коллизии открытая адресация или закрытое хеширование вычисляет новые позиции с помощью последовательности зондов, и следующая запись сохраняется в этой позиции. В этом методе все значения сохраняются в хеш-таблице. Хеш-таблица содержит два типа значений: контрольные значения (например, -1) и значения данных. Наличие контрольного значения указывает на то, что местоположение в данный момент не содержит значения данных, но может использоваться для хранения значения. Когда ключ сопоставляется с определенным местоположением памяти, то проверяется хранящееся в нем значение. Если он содержит контрольное значение, то местоположение свободно, и в нем можно сохранить значение данных. Однако если местоположение уже имеет некоторое сохраненное значение данных, то другие слоты систематически проверяются в прямом направлении, чтобы найти свободный слот. Если не найдено ни одного свободного места, то возникает состояние OVERFLOW. Процесс проверки мест памяти в хэш-таблице называется зондированием. Метод открытой адресации может быть реализован с использованием линейного зондирования, квадратичного зондирования, двойного хеширования и повторного хеширования.



Линейное зондирование

Самый простой подход к разрешению коллизии — линейное зондирование. В этом методе, если значение уже сохранено в месте, сгенерированном $h(k)$, то для разрешения коллизии используется следующая хеш-функция:

$$h(k, i) = [h'(k) + i] \bmod m$$

Где m — размер хэш-таблицы, $h'(k) = (k \bmod m)$, а i — номер зонда, который изменяется от 0 до $m - 1$. Следовательно, для заданного ключа k сначала зондируется место, сгенерированное $[h'(k) \bmod m]$, поскольку в первый раз $i = 0$. Если ячейка свободна, значение сохраняется в ней, в противном случае второй зонд генерирует адрес ячейки, заданный как $[h'(k) + 1] \bmod m$. Аналогично, если ячейка занята, то последующие зонды генерируют адрес как $[h'(k) + 2] \bmod m$, $[h'(k) + 3] \bmod m$, $[h'(k) + 4] \bmod m$, $[h'(k) + 5] \bmod m$ и так далее, пока не будет найдена свободная ячейка.

Примечание.

Линейное зондирование известно своей простотой. Когда нам нужно сохранить значение, мы пробуем слоты: $[h'(k)] \bmod m$, $[h'(k) + 1] \bmod m$, $[h'(k) + 2] \bmod m$, $[h'(k) + 3] \bmod m$, $[h'(k) + 4] \bmod m$, $[h'(k) + 5] \bmod m$ и так далее, пока не будет найдено свободное место.



Линейное зондирование - пример

пример

Рассмотрим хеш-таблицу размером 10. Используя линейное зондирование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Пусть $h'(k) = k \bmod m$, $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Шаг 1: Ключ = 72

$$h(72, 0) = (72 \% 10 + 0) \% 10 = 2 \% 10 = 2$$

Так как $T[2]$ свободен, вставьте ключ 72 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Шаг 2: Ключ = 27

$$h(27, 0) = (27 \% 10 + 0) \% 10 = 7 \% 10 = 7$$

Так как $T[7]$ свободен, вставьте ключ 27 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Линейное зондирование - пример

пример

Рассмотрим хеш-таблицу размером 10. Используя линейное зондирование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Пусть $h'(k) = k \bmod m$, $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Шаг 3: Ключ = 36

$$h(36, 0) = (36 \% 10 + 0) \% 10 = 6 \% 10 = 6$$

Так как $T[6]$ свободен, вставьте ключ 36 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Шаг 4: Ключ = 24

$$h(24, 0) = (24 \% 10 + 0) \% 10 = 4 \% 10 = 4$$

Так как $T[4]$ свободен, вставьте ключ 24 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Линейное зондирование - пример

пример

Рассмотрим хеш-таблицу размером 10. Используя линейное зондирование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Пусть $h'(k) = k \bmod m$, $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Шаг 5: Ключ = 63

$$h(63, 0) = (63 \% 10 + 0) \% 10 = 3 \% 10 = 3$$

Так как $T[3]$ свободен, вставьте ключ 63 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Шаг 6: Ключ = 81

$$h(81, 0) = (81 \% 10 + 0) \% 10 = 1 \% 10 = 1$$

Так как $T[1]$ свободен, вставьте ключ 81 в это место.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Линейное зондирование - пример

пример

Рассмотрим хеш-таблицу размером 10. Используя линейное зондирование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Пусть $h'(k) = k \bmod m$, $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Шаг 7: Ключ = 92

$$h(92, 0) = (92 \% 10 + 0) \% 10 = 2 \% 10 = 2$$

Теперь $T[2]$ занято, поэтому мы не можем сохранить ключ 92 в $T[2]$. Поэтому попробуйте еще раз для следующего местоположения. Таким образом, на этот раз зонд, $i = 1$.

$$h(92, 1) = (92 \% 10 + 1) \% 10 = (2 + 1) \% 10 = 3$$

Теперь $T[3]$ занято, поэтому мы не можем сохранить ключ 92 в $T[3]$. Поэтому попробуйте еще раз для следующего местоположения. Таким образом, на этот раз зонд, $i = 2$.

$$h(92, 2) = (92 \% 10 + 2) \% 10 = (2 + 2) \% 10 = 4$$

Теперь $T[4]$ занято, поэтому мы не можем сохранить ключ 92 в $T[4]$. Поэтому попробуем снова для следующего местоположения. Таким образом, на этот раз зондируем $i = 3$.

$$h(92, 3) = (92 \% 10 + 3) \% 10 = (2 + 3) \% 10 = 5$$

Так как $T[5]$ свободен, вставьте ключ 92 в это место.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	92	36	27	-1	-1

Линейное зондирование - пример

пример

Рассмотрим хеш-таблицу размером 10. Используя линейное зондирование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Пусть $h'(k) = k \bmod m$, $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Шаг 8: Ключ = 101

$$h(101, 0) = (101 \% 10 + 0) \% 10 = 1 \% 10 = 1$$

Теперь $T[1]$ занято, поэтому мы не можем сохранить ключ 101 в $T[1]$. Поэтому попробуйте еще раз для следующего местоположения. Таким образом, на этот раз зонд, $i = 1$.

$$h(101, 1) = (101 \% 10 + 1) \% 10 = (2 + 1) \% 10 = 2$$

$T[2]$ также занято, поэтому мы не можем сохранить ключ в этом расположении. Процедура будет повторяться до тех пор, пока хэш-функция не сгенерирует адрес ячейки 8, которая является свободной и может использоваться для хранения в ней значения.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	92	36	27	101	-1

Линейное зондирование

Поиск значения в хэш-таблице

Время поиска в этом случае задается как $O(1)$. Если ключ не совпадает, то функция поиска начинает последовательный поиск массива, который продолжается до тех пор, пока:

- значение не будет найдено, или
- функция поиска не встретит свободную ячейку в массиве, что указывает на то, что значение отсутствует, или
- функция поиска завершается, поскольку достигает конца таблицы, а значение отсутствует.

В худшем случае операция поиска может потребовать выполнения $n-1$ сравнений, а время выполнения алгоритма поиска может занять $O(n)$ времени.

Плюсы и минусы

Линейное зондирование находит пустое место, выполняя линейный поиск в массиве, начиная с позиции $h(k)$. Хотя алгоритм обеспечивает хорошее кэширование памяти за счет хорошей локальности ссылок, недостатком этого алгоритма является то, что он приводит к кластеризации, и, таким образом, существует более высокий риск большего количества коллизий, где одно столкновение уже произошло.

Квадратичное зондирование

В этой технике, если значение уже сохранено в месте, сгенерированном $h(k)$, то для разрешения коллизии используется следующая хэш-функция:

$$h(k, i) = [h(k) + c_1 i + c_2 i^2] \bmod m$$

где m — размер хэш-таблицы, $h'(k) = (k \bmod m)$, i — номер зонда, который изменяется от 0 до $m-1$, а c_1 и c_2 — константы, такие, что c_1 и $c_2 \neq 0$. Квадратичное зондирование устраняет первичное явление кластеризации линейного зондирования, поскольку вместо выполнения линейного поиска выполняется квадратичный поиск. Для заданного ключа k сначала зондируется место, сгенерированное $h'(k) \bmod m$. Если место свободно, значение сохраняется в нем, в противном случае последующие зондируемые места смещаются на коэффициенты, которые зависят квадратичным образом от номера зонда i . Хотя квадратичное зондирование работает лучше, чем линейное, для максимального использования хэш-таблицы необходимо ограничить значения c_1 , c_2 и m .



Квадратичное зондирование - пример

пример

Рассмотрим хеш-таблицу размером 10. Используя квадратичное зондирование, вставьте ключи 72, 27, 36, 24, 63, 81 и 101 в таблицу. Возьмем $c_1 = 1$ и $c_2 = 3$. Решение Пусть $h'(k) = k \bmod m$, $m = 10$ Первоначально хеш-таблица может быть задана как:

Имеем,

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

Шаг 1: Ключ = 72

$$h(72, 0) = (72 \% 10 + 1 \times 0 + 3 \times 0) \% 10 = 2 \% 10 = 2$$

Так как $T[2]$ свободен, вставьте ключ 72 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Шаг 2: Ключ = 27

$$h(27, 0) = (27 \% 10 + 1 \times 0 + 3 \times 0) \% 10 = 7 \% 10 = 7$$

Так как $T[7]$ свободен, вставьте ключ 27 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Квадратичное зондирование - пример

пример

Рассмотрим хеш-таблицу размером 10. Используя квадратичное зондирование, вставьте ключи 72, 27, 36, 24, 63, 81 и 101 в таблицу. Возьмем $c_1 = 1$ и $c_2 = 3$.
Решение Пусть $h'(k) = k \bmod m$, $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Шаг 3: Ключ = 36

$$h(36, 0) = (36 \% 10 + 1 \times 0 + 3 \times 0) \% 10 = 6 \% 10 = 6$$

Так как $T[6]$ свободен, вставьте ключ 36 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Шаг 4: Ключ = 24

$$h(24, 0) = (24 \% 10 + 1 \times 0 + 3 \times 0) \% 10 = 4 \% 10 = 4$$

Так как $T[4]$ свободен, вставьте ключ 24 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Квадратичное зондирование - пример

пример

Рассмотрим хеш-таблицу размером 10. Используя квадратичное зондирование, вставьте ключи 72, 27, 36, 24, 63, 81 и 101 в таблицу. Возьмем $c_1 = 1$ и $c_2 = 3$.

Решение Пусть $h'(k) = k \bmod m$, $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Шаг 5: Ключ = 63

$$h(63, 0) = (63 \% 10 + 1 \times 0 + 3 \times 0) \% 10 = 3 \% 10 = 3$$

Так как $T[3]$ свободен, вставьте ключ 63 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Шаг 6: Ключ = 81

$$h(81, 0) = (81 \% 10 + 1 \times 0 + 3 \times 0) \% 10 = 1 \% 10 = 1$$

Так как $T[1]$ свободен, вставьте ключ 81 в это место.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Квадратичное зондирование - пример

пример

Рассмотрим хеш-таблицу размером 10. Используя квадратичное зондирование, вставьте ключи 72, 27, 36, 24, 63, 81 и 101 в таблицу. Возьмем $c_1 = 1$ и $c_2 = 3$.

Решение Пусть $h'(k) = k \bmod m$, $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Шаг 7: Ключ = 101

$$h(101, 0) = (101 \% 10 + 1 \times 0 + 3 \times 0) \% 10 = 1 \% 10 = 1$$

Теперь $T[1]$ занято, поэтому мы не можем сохранить ключ 92 в $T[2]$. Поэтому попробуйте еще раз для следующего местоположения. Таким образом, на этот раз зонд, $i = 1$.

$$h(101, 1) = (101 \% 10 + 1 \times 1 + 3 \times 1) \% 10 = (1 + 1 + 3) \% 10 = 5$$

Поскольку $T[5]$ пуст, вставьте ключ 101 в $T[5]$.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	101	36	27	-1	-1

Квадратичное зондирование

Поиск Если желаемое значение ключа совпадает со значением ключа в этом месте, то элемент присутствует в хэш-таблице, и поиск считается успешным. В этом случае время поиска задается как $O(1)$. Однако если значение не совпадает, то функция поиска начинает последовательный поиск в массиве, который продолжается до тех пор, пока:

- значение не будет найдено, или
- функция поиска не встретит пустое место в массиве, что указывает на то, что значение отсутствует, или
- функция поиска завершает работу, поскольку достигает конца таблицы, а значение отсутствует.

В худшем случае операция поиска может потребовать $n-1$ сравнений, а время выполнения алгоритма поиска может быть $O(n)$. Худший случай возникнет, когда после сканирования всех $n-1$ элементов значение либо присутствует в последнем месте, либо отсутствует в таблице.

Плюсы и минусы Квадратичный зонд решает основную проблему кластеризации, которая существует в методе линейного зондирования. Квадратичный зонд обеспечивает хорошее кэширование памяти, поскольку сохраняет некоторую локальность ссылок. Но линейное зондирование лучше справляется с этой задачей и обеспечивает лучшую производительность кэша. Одним из основных недостатков квадратичного зондирования - последовательность последовательных зондирований может исследовать только часть таблицы, и эта часть может быть довольно маленькой. Если это произойдет, то мы не сможем найти пустое место в таблице несмотря на то, что таблица никоим образом не заполнена.

Двойное хеширование

Для начала, двойное хеширование использует одно хеш-значение, а затем многократно переходит на интервал вперед, пока не будет достигнуто пустое место. Интервал определяется с помощью второй, независимой хеш-функции, отсюда и название двойное хеширование. В двойном хешировании мы используем две хеш-функции, а не одну. Хэш-функция в случае двойного хеширования может быть задана как:

$$h(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

где m — размер хеш-таблицы, $h_1(k)$ и $h_2(k)$ — две хеш-функции, заданные как $h_1(k) = k \bmod m$, $h_2(k) = k \bmod m'$, i — номер пробы, который изменяется от 0 до $m-1$, а m' выбирается меньше m . Мы можем выбрать $m' = m-1$ или $m-2$. Когда нам нужно вставить ключ k в хеш-таблицу, мы сначала проверяем местоположение, заданное применением $[h_1(k) \bmod m]$, потому что во время первого зондирования $i = 0$. Если местоположение свободно, ключ вставляется в него, в противном случае последующие зондирования генерируют местоположения, которые находятся со смещением $[h_2(k) \bmod m]$ от предыдущего местоположения. Поскольку смещение может меняться с каждым зондом в зависимости от значения, сгенерированного второй хеш-функцией, производительность двойного хеширования очень близка к производительности идеальной схемы равномерного хеширования.

Двойное хеширование - пример

пример

Рассмотрим хеш-таблицу размером = 10. Используя двойное хеширование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Возьмем $h_1 = (k \bmod 10)$ и $h_2 = (k \bmod 8)$. Пусть $m = 10$

Изначально хеш-таблица может быть задана как:

Имеем,

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

$$h(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

Шаг 1: Ключ = 72

$$h(72, 0) = (72 \% 10 + 0 \times 72 \% 8) \% 10 = 2 \% 10 = 2$$

Так как $T[2]$ свободен, вставьте ключ 72 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Шаг 2: Ключ = 27

$$h(27, 0) = (27 \% 10 + 1 \times 0 + 0 \times 27 \% 8) \% 10 = 7 \% 10 = 7$$

Так как $T[7]$ свободен, вставьте ключ 27 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Двойное хеширование - пример

пример

Рассмотрим хеш-таблицу размером = 10. Используя двойное хеширование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Возьмем $h_1 = (k \bmod 10)$ и $h_2 = (k \bmod 8)$. Пусть $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Шаг 3: Ключ = 36

$$h(36, 0) = (36 \% 10 + 0 \times 36 \% 8) \% 10 = 6 \% 10 = 6$$

Так как $T[6]$ свободен, вставьте ключ 36 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Шаг 4: Ключ = 24

$$h(24, 0) = (24 \% 10 + 0 \times 24 \% 8) \% 10 = 4 \% 10 = 4$$

Так как $T[4]$ свободен, вставьте ключ 24 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Двойное хеширование - пример

пример

Рассмотрим хеш-таблицу размером = 10. Используя двойное хеширование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Возьмем $h_1 = (k \bmod 10)$ и $h_2 = (k \bmod 8)$. Пусть $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Шаг 5: Ключ = 63

$$h(63, 0) = (63 \% 10 + 0 \times 63 \% 8) \% 10 = 3 \% 10 = 3$$

Так как $T[3]$ свободен, вставьте ключ 63 в это место.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Шаг 6: Ключ = 81

$$h(81, 0) = (81 \% 10 + 0 \times 81 \% 8) \% 10 = 1 \% 10 = 1$$

Так как $T[1]$ свободен, вставьте ключ 81 в это место.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Двойное хеширование - пример

пример

Рассмотрим хеш-таблицу размером = 10. Используя двойное хеширование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Возьмем $h_1 = (k \bmod 10)$ и $h_2 = (k \bmod 8)$. Пусть $m = 10$

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Шаг 7: Ключ = 92

$$h(92, 0) = (92 \% 10 + 0 \times 92 \% 8) \% 10 = 2 \% 10 = 2$$

Теперь $T[2]$ занято, поэтому мы не можем сохранить ключ 92 в $T[2]$. Поэтому попробуйте еще раз для следующего местоположения. Таким образом, на этот раз зонд, $i = 1$.

$$h(92, 1) = (92 \% 10 + 1 \times 92 \% 8) \% 10 = (2 + 4) \% 10 = 6$$

Теперь $T[6]$ занято, поэтому мы не можем сохранить ключ 92 в $T[6]$. Поэтому попробуйте еще раз для следующего местоположения. Таким образом, на этот раз зонд, $i = 2$.

Так как $T[0]$ свободен, вставьте ключ 92 в это место.

0	1	2	3	4	5	6	7	8	9
92	81	72	63	24	-1	36	27	-1	-1

Двойное хеширование - пример

пример

Рассмотрим хеш-таблицу размером = 10. Используя двойное хеширование, вставьте ключи 72, 27, 36, 24, 63, 81, 92 и 101 в таблицу. Возьмем $h_1 = (k \bmod 10)$ и $h_2 = (k \bmod 8)$. Пусть $m = 10$

0	1	2	3	4	5	6	7	8	9
92	81	72	63	24	-1	36	27	-1	-1

Шаг 8: Ключ = 101

$$h(101, 0) = (101 \% 10 + 0 \times 101 \% 8) \% 10 = 1 \% 10 = 1$$

Теперь $T[1]$ занято, поэтому мы не можем сохранить ключ 101 в $T[1]$. Поэтому попробуйте еще раз для следующего местоположения. Таким образом, на этот раз зонд, $i = 1$.

$$h(101, 1) = (101 \% 10 + 1 \times 101 \% 8) \% 10 = (1 + 5) \% 10 = 6$$

$T[6]$ также занято, поэтому мы не можем сохранить ключ в этом расположении. Вы увидите, что нам нужно выполнить зондирование много раз, чтобы вставить ключ 101 в хэш-таблицу. Хотя двойное хэширование является очень эффективным алгоритмом, оно всегда требует, чтобы m было простым числом. В нашем случае $m=10$, что не является простым числом, отсюда и ухудшение производительности. Если бы m было равно 11, алгоритм работал бы очень эффективно. Таким образом, можно сказать, что производительность метода чувствительна к значению m .

Плюсы и минусы

Двойное хеширование минимизирует повторяющиеся коллизии и эффекты кластеризации. То есть двойное хеширование свободно от проблем, связанных как с первичной кластеризацией, так и со вторичной кластеризацией.



Перехеширование

Когда хеш-таблица почти заполнена, количество коллизий увеличивается, тем самым снижая производительность операций вставки и поиска. В таких случаях лучшим вариантом будет создание новой хеш-таблицы с размером, вдвое превышающим размер исходной хеш-таблицы. Затем все записи в исходной хеш-таблице придется переместить в новую хеш-таблицу. Это делается путем взятия каждой записи, вычисления ее нового значения хеш-функции и последующей вставки ее в новую хеш-таблицу. Хотя перехеширование кажется простым процессом, оно довольно затратно и поэтому не должно выполняться часто. Рассмотрим хеш-таблицу размером 5, приведенную ниже. Используемая хеш-функция — $h(x) = x \% 5$. Перехешируем записи в новую хеш-таблицу.

0	1	2	3	4
	26	31	43	17

Обратите внимание, что новая хеш-таблица состоит из 10 ячеек, что вдвое больше размера исходной таблицы.

0	1	2	3	4	5	6	7	8	9

Теперь перехешируем ключевые значения из старой хеш-таблицы в новую, используя хеш-функцию — $h(x) = x \% 10$

0	1	2	3	4	5	6	7	8	9
	31		43			26	17		

Анализ хеширования с открытой адресацией

Анализ открытой адресации, как и анализ метода цепочек, выполняется с использованием коэффициента заполнения $\alpha = n/m$. Само собой разумеется, при использовании открытой адресации может быть не более одного элемента на ячейку таблицы, так что $n \leq m$ и, следовательно, $\alpha \leq 1$.

Будем считать, что используется равномерное хеширование. При такой идеализированной схеме последовательность исследований $\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle$, используемая для вставки или поиска каждого ключа k , с равной вероятностью является одной из возможных перестановок $(0, 1, \dots, m - 1)$. Разумеется, с каждым конкретным ключом связана единственная фиксированная последовательность исследований, так что при рассмотрении распределения вероятностей ключей и хеш-функций все последовательности исследований оказываются равновероятными.

Мы проанализируем математическое ожидание количества исследований для хеширования с открытой адресацией в предположении равномерного хеширования и начнем с анализа количества исследований в случае неудачного поиска.

Анализ хеширования с открытой адресацией

Теорема 12.1

Математическое ожидание количества исследований при неудачном поиске в хеш-таблице с открытой адресацией и коэффициентом заполнения $\alpha = n/m < 1$ в предположении равномерного хеширования не превышает $1/(1 - \alpha)$.

Доказательство

Неудачный поиск: Каждая последовательность исследований завершается пустой ячейкой. Пусть X – количество исследований, выполненных при неудачном поиске.

События ($i = 1, 2, \dots$) пришлись на занятую ячейку. Тогда событие $\{X \geq i\}$ представляет собой пересечение событий $A_1 \cap A_2 \cap \dots \cap A_{i-1}$. Ограничим вероятность $\Pr\{X \geq i\}$ путем ограничения вероятности $\Pr\{A_1 \cap A_2 \cap \dots \cap A_{i-1}\}$.

$$\Pr\{A_1 \cap A_2 \cap \dots \cap A_{i-1}\} = \Pr\{A_1\} \cdot \Pr\{A_2 | A_1\} \cdot \Pr\{A_3 | A_1 \cap A_2\} \cdot \dots \cdot \Pr\{A_{i-1} | A_1 \cap A_2 \cap \dots \cap A_{i-2}\}.$$

Имеется n элементов и m ячеек, $\Pr\{A_1\} = n/m$. Вероятность того, что будет выполнено j -е исследование ($j > 1$) и что оно будет проведено над заполненной ячейкой (при этом первые $j - 1$ исследований проведены над заполненными ячейками), равна $(n - j + 1)/(m - j + 1)$. **Факт:** из $n < m$ для всех $0 \leq j < m$ следует соотношение $(n - j)/(m - j) \leq n/m$, для всех $1 \leq i \leq m$ получаем

$$\Pr\{X \geq i\} = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \dots \frac{n-i-2}{m-i-2} \leq \left(\frac{n}{m}\right)^{i-1} = \alpha^{i-1}$$

Анализ хеширования с открытой адресацией

Доказательство (продолжение)

Теперь воспользуемся уравнением для получения границы ожидаемого количества исследований:

$$E[X] = \sum_{i=1}^{\infty} \Pr\{X \geq i\} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \sum_{i=0}^{\infty} \alpha^i = \frac{1}{1 - \alpha}$$

Полученная граница $1/(1 - \alpha) = 1 + \alpha + \alpha^2 + \dots$ имеет интуитивную интерпретацию. Одно исследование выполняется всегда. С вероятностью, приблизительно равной α , первое исследование проводится над заполненной ячейкой, и требуется выполнение второго исследования. С вероятностью, приблизительно равной α^2 , две первые ячейки оказываются заполненными, и требуется проведение третьего исследования, и т.д.

Если α - константа, то неудачный поиск выполняется за время $O(1)$. Например, $1/(1 - .5) = 2$. При заполненности хеш-таблицы на 90% - $1/(1 - .9) = 10$. Теорема практически непосредственно дает оценку производительности процедуры Hash-Insert.

Следствие 12.1

Вставка элемента в хеш-таблицу с открытой адресацией и коэффициентом заполнения α в предположении равномерного хеширования требует в среднем не более $1/(1 - \alpha)$ исследований.

Анализ хеширования с открытой адресацией

Теорема 12.2

Математическое ожидание количества исследований при удачном поиске в хеш-таблице с открытой адресацией и коэффициентом заполнения $\alpha < 1$, в предположении равномерного хеширования и равновероятного поиска любого из ключей, не превышает

$$\frac{1}{\alpha} \ln \frac{1}{1 - \alpha}$$

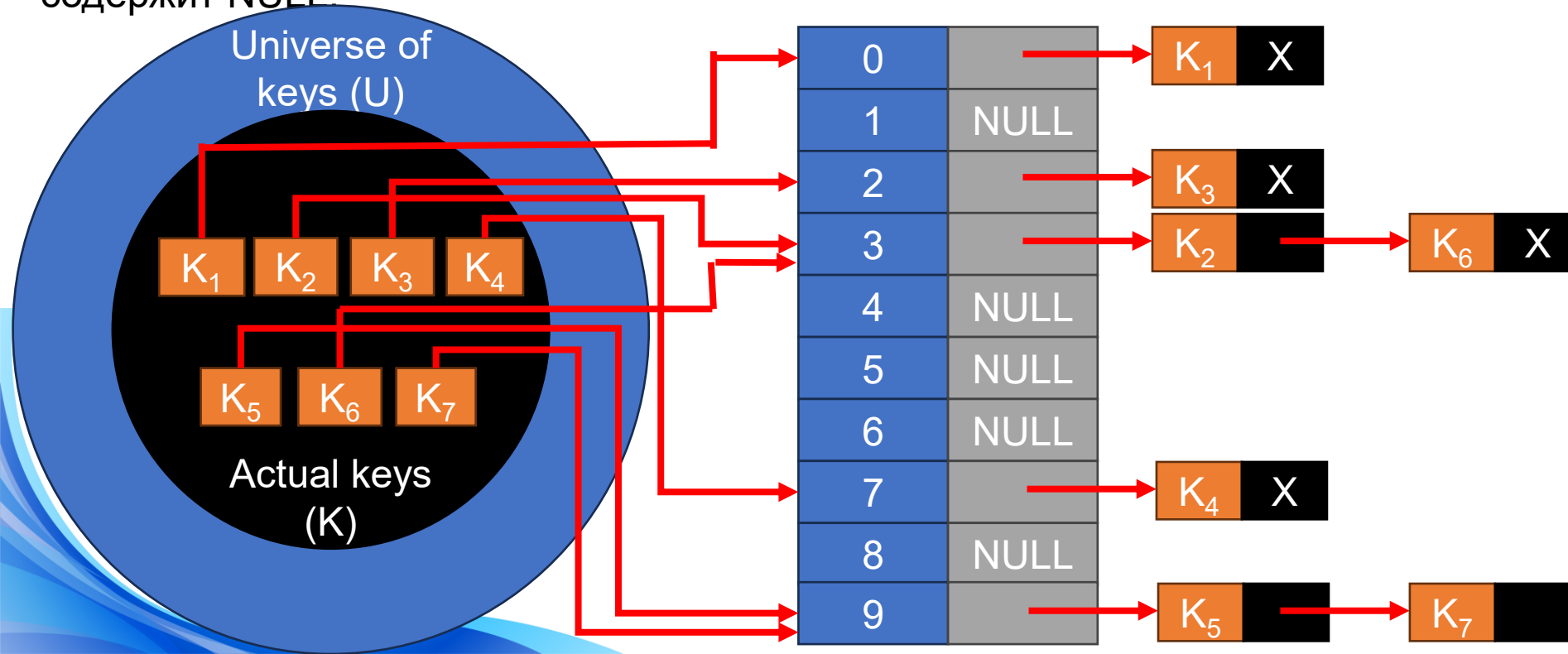
Доказательство. Поиск ключа k выполняется той же последовательностью исследований, что и его вставка. В соответствии со следствием 12.1, если k был $(i + 1)$ -м ключом, вставленным в хеш-таблицу, то математическое ожидание количества проб при поиске k не превышает $1/(1 - i/m) = m/(m - i)$. Усреднение по всем n ключам в хеш-таблице дает нам среднее количество исследований при удачном поиске:

$$\frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} = \frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k} \leq \frac{1}{\alpha} \int_{m-n}^m \frac{1}{x} dx = \frac{1}{\alpha} \ln \frac{m}{m-n} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$$

В наполовину заполненной хеш-таблице ожидаемое количество исследований при удачном поиске оказывается меньше 1.387. Если хеш-таблица заполнена на 90%, ожидаемое количество исследований меньше 2.559.

Разрешение коллизий путем объединения в цепочку

При объединении в цепочку каждое местоположение в хэш-таблице хранит указатель на связанный список, содержащий все значения ключей, которые были хэшированы в это местоположение. То есть местоположение i в хэш-таблице указывает на заголовок связанного списка всех значений ключей, которые хэшированы в i . Однако если ни одно значение ключа не хэшируется в i , то местоположение i в хэш-таблице содержит NULL.



```
//Structure of the node
typedef struct node_HT
{
    int value;
    struct node *next;
} node;
```

Код для инициализации цепной хэш-таблицы

```
/* Initializes m location in the chained hash table.  
The operation takes a running time of O(m) */  
void initializeHashTable(node *hash_table[],  
                          int m)  
{  
    int i;  
    for (i = 0; i <= m; i++)  
        hash_table[i] = NULL;  
}
```

Код для вставки значения

```
/* The element is inserted at the beginning of
the linked list whose pointer to its head is
stored in the location given by h(k). The running
time of the insert operation is  $O(1)$ , as
the new key value is always added as the first
element of the list irrespective of the size of
the linked list as well as that of the chained
hash table. */
node *insert_value(node *hash_table[], int val)
{
    node *new_node;
    new_node = (node *)malloc(sizeof(node));
    new_node->value = val;
    new_node->next = hash_table[h(val)];
    hash_table[h(val)] = new_node;
}
```

Код для поиска значения

```
/* The element is searched in the linked
list whose pointer to its head is stored
in the location given by h(k). If search is
successful, the function returns a pointer
to the node in the linked list; otherwise
it returns NULL. The worst case running
time of the search operation is given as
order of size of the linked list. */
node *search_value(node *hash_table[], int val)
{
    node *ptr;
    ptr = hash_table[h(val)];
    while ((ptr != NULL) && (ptr->value != val))
        ptr = ptr->next;
    if (ptr->value == val)
        return ptr;
    else
        return NULL;
}
```

Код для удаления значения

/* To delete a node from the linked list whose head is stored at the location given by h(k) in the hash table, we need to know the address of the node's predecessor. We do this using a pointer save. The running time complexity of the delete operation is same as that of the search operation because we need to search the predecessor of the node so that the node can be removed without affecting other nodes in the list. */

```
void delete_value(node *hash_table[], int val) {  
    node *save, *ptr;  
    save = NULL;  
    ptr = hash_table[h(val)];  
    while ((ptr != NULL) && (ptr->value != val)) {  
        save = ptr;  
        ptr = ptr->next;  
    }  
    if (ptr != NULL) {  
        save->next = ptr->next;  
        free(ptr);  
    }  
    else  
        printf("\n VALUE NOT FOUND");  
}
```


Разрешение коллизий путем объединения в цепочку - пример

пример

Вставьте ключи 7, 24, 18, 52, 36, 54, 11 и 23 в цепочку хэш-таблицы из 9 ячеек памяти. Используйте $h(k) = k \bmod m$. В этом случае $m=9$. Изначально хэш-таблица может быть задана как:

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL

Шаг 1

Ключ = 7
 $h(k) = 7 \bmod 9 = 7$
Создайте связанный список для ячейки 7 и сохраните в нем ключевое значение 7 как его единственный узел.

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	→ 7 X
8	NULL

Шаг 2

Ключ = 24
 $h(k) = 24 \bmod 9 = 6$
Создайте связанный список для ячейки 6 и сохраните в нем ключевое значение 24 как его единственный узел.

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	→ 24 X
7	→ 7 X
8	NULL

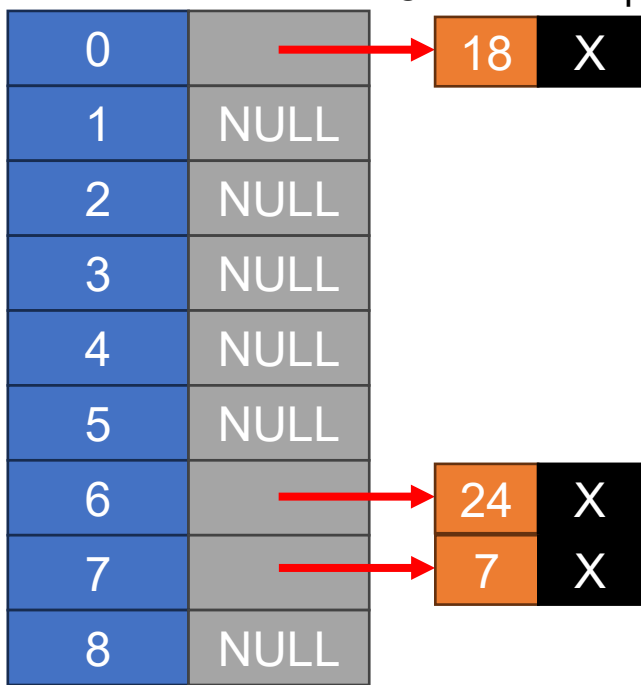
Разрешение коллизий путем объединения в цепочку - пример

пример 7, 24, 18, 52, 36, 54, 11 и 23

Шаг 3

Ключ = 18 $h(k) = 18 \bmod 9 = 0$

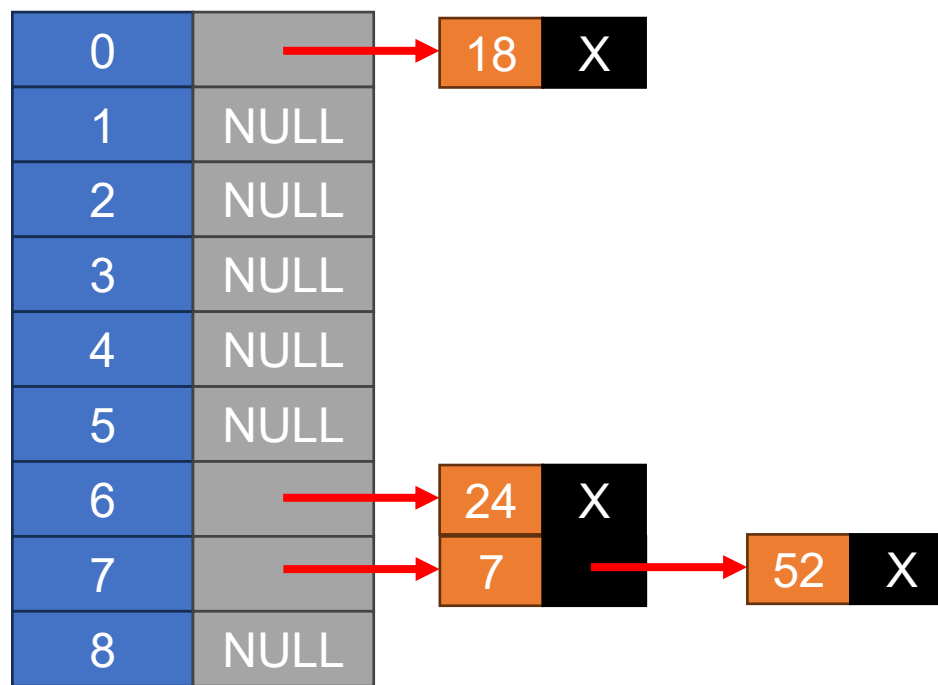
Создайте связанный список для местоположения 0 и сохраните в нем значение ключа 18 как его единственный узел.



Шаг 4

Ключ = 52 $h(k) = 52 \bmod 9 = 7$

Вставьте 52 в конец связанного списка местоположения 7.



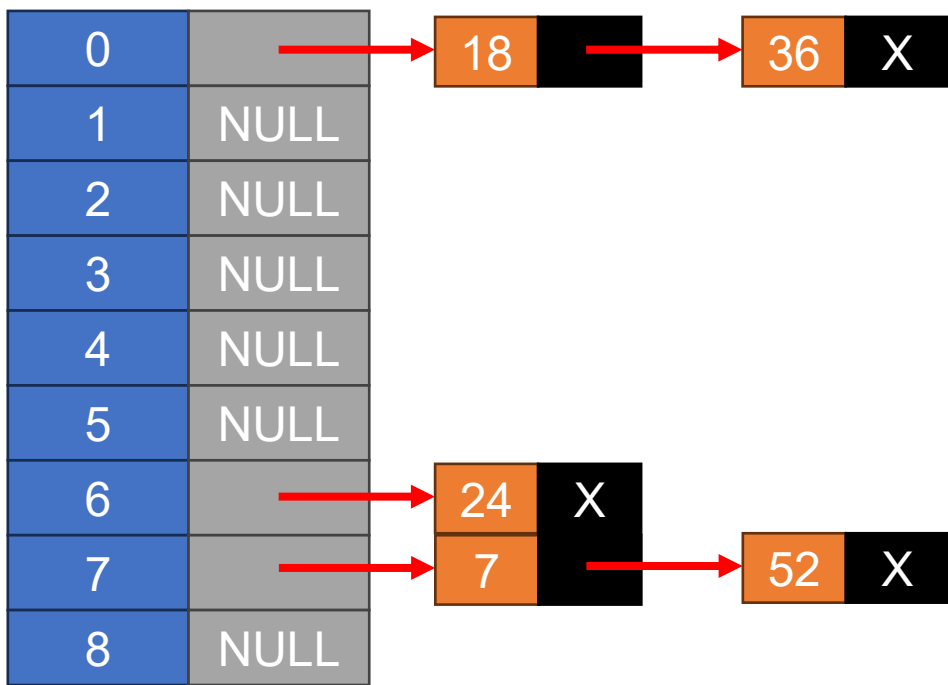
Разрешение коллизий путем объединения в цепочку - пример

пример 7, 24, 18, 52, 36, 54, 11 и 23

Шаг 5:

Ключ = 36 $h(k) = 36 \bmod 9 = 0$

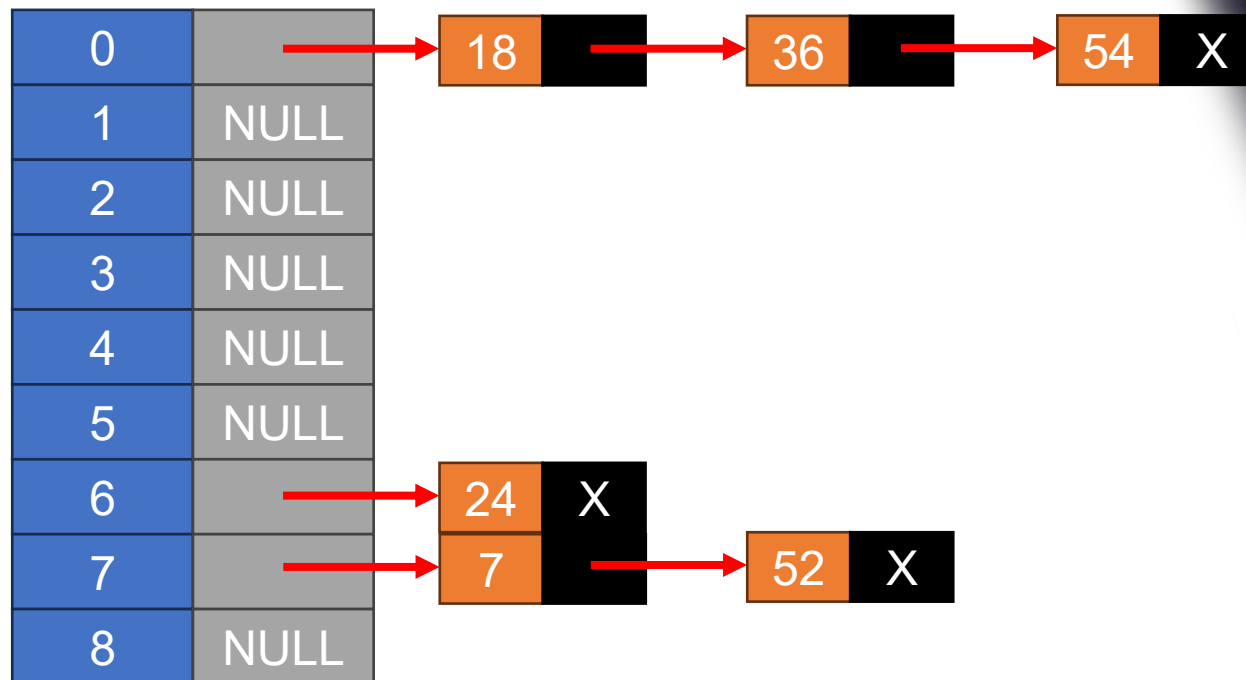
Вставьте 36 в конец связанного списка местоположения 0.



Шаг 6:

Ключ = 54 $h(k) = 54 \bmod 9 = 0$

Вставьте 54 в конец связанного списка местоположения 0.



Разрешение коллизий путем объединения в цепочку - пример

пример 7, 24, 18, 52, 36, 54, 11 и 23

Шаг 7:

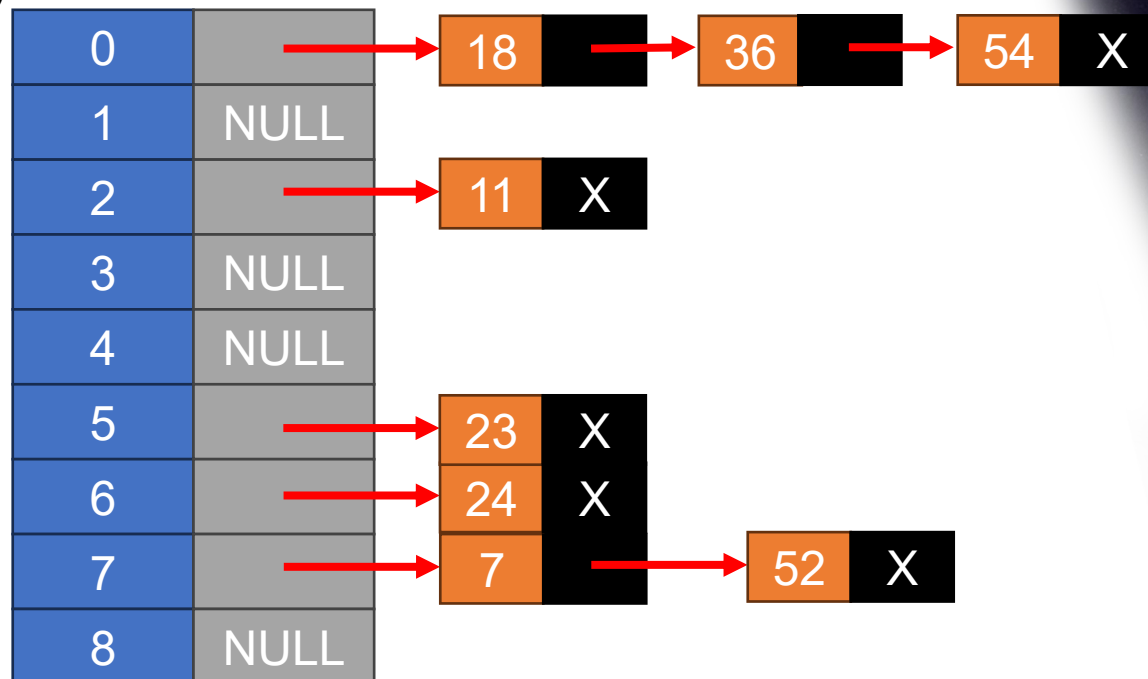
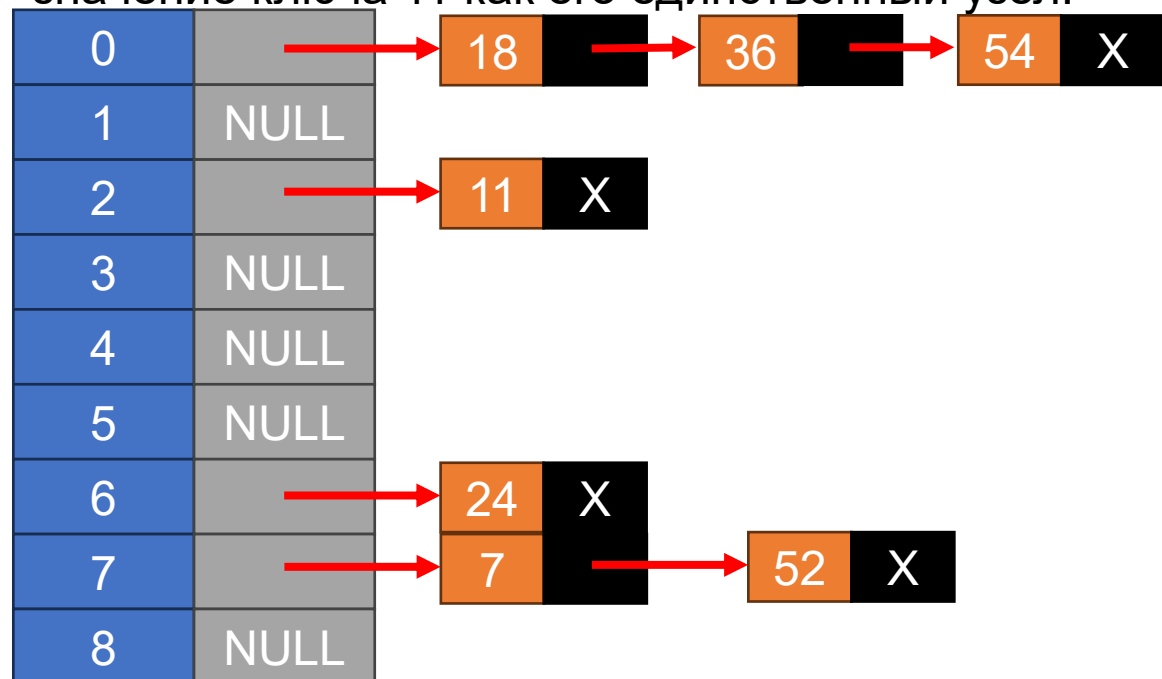
Ключ = 11 $h(k) = 11 \bmod 9 = 2$

Создайте связанный список для местоположения 2 и сохраните в нем значение ключа 11 как его единственный узел.

Шаг 8:

Ключ = 23 $h(k) = 23 \bmod 9 = 5$

Создайте связанный список для местоположения 5 и сохраните в нем значение ключа 23 как его единственный узел.



Анализ хеширования с цепочками

Насколько высока производительность хеширования с цепочками? В частности, сколько времени требуется для поиска элемента с заданным ключом?

Пусть: хеш-таблица T с m ячейками, в которых хранятся n элементов. Определим коэффициент заполнения таблицы T как n/m , т.е. как среднее количество элементов, хранящихся в одной цепочке. Наш анализ будет опираться на значение величины α , которая может быть меньше, равна или больше единицы.

В наихудшем случае хеширование с цепочками: все n ключей хешированы в одну и ту же ячейку, создав список длиной n . Время поиска в наихудшем случае равно $\Theta(n) + \Theta(t)$.

Понятно, что использование хеш-таблиц в наихудшем случае совершенно бессмысленно.

Производительность хеширования в среднем случае зависит от того, насколько хорошо хеш-функция h распределяет множество сохраняемых ключей по m ячейкам в среднем. Будем полагать, что все элементы хешируются по ячейкам равномерно и независимо, и назовем данное предположение **простым равномерным хешированием** (simple uniform hashing).

Обозначим длины списков $T[j]$ для $j = 0, 1, \dots, m - 1$ как n_j , так что

$$n = n_0 + n_1 + \dots + n_{m-1}$$

а ожидаемое значение n , равно $E[n] = \alpha = n/m$.

Анализ хеширования с цепочками

Пусть время вычисления хеш-значения $h(k)$ - $O(1)$, время, необходимое для поиска элемента с ключом k , линейно зависит от длины $n_{h(k)}$ списка $T[h(k)]$. Рассмотрим математическое ожидание количества элементов, которое должно быть проверено алгоритмом поиска (т.е. количество элементов в списке $T[h(k)]$, которые проверяются на равенство их ключей величине k). Два случая: когда поиск неудачен и в таблице нет элементов с ключом k ; поиск заканчивается успешно и в таблице определяется элемент с ключом k .

Теорема 12.3

В хеш-таблице с разрешением коллизий методом цепочек время неудачного поиска в среднем случае в предположении простого равномерного хеширования составляет $\Theta(1 + \alpha)$.

Доказательство.

В предположении простого равномерного хеширования любой ключ k , который еще не находится в таблице, может быть помещен с равной вероятностью в любую из m ячеек. Математическое ожидание времени неудачного поиска ключа k равно времени поиска до конца списка $T[h(k)]$, ожидаемая длина которого – $E[n_{h(k)}] = \alpha$. Таким образом, при неудачном поиске математическое ожидание количества проверяемых элементов равно α , а общее время, необходимое для поиска, включая время вычисления хеш-функции $h(k)$, равно $O(1 + \alpha)$. Успешный поиск несколько отличается от неудачного, поскольку вероятность поиска в списке различна для разных списков и пропорциональна количеству содержащихся в нем элементов. Математическое ожидание времени поиска остается равным $\Theta(1 + \alpha)$.

Анализ хеширования с цепочками

Теорема 12.4

В хеш-таблице с разрешением коллизий методом цепочек время успешного поиска в среднем случае в предположении простого равномерного хеширования в среднем равно $\Theta(1 + \alpha)$.

Доказательство.

Пусть x_i обозначает i -й элемент, вставленный в таблицу ($i = 1, 2, \dots, n$), и пусть $k_i = x_i.\text{key}$. Определим для ключей k_i и k_j , индикаторную случайную величину $X_{ij} = I\{h(k_i) = h(k_j)\}$. В предположении простого равномерного хеширования $\Pr\{h(k_i) = h(k_j)\} = 1/m$ и, соответственно, $E[X_{ij}] = 1/m$. Таким образом, математическое ожидание количества проверяемых элементов в случае успешного поиска равно

$$\begin{aligned} E \left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \right] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E[X_{ij}] \right) = \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) = 1 + \frac{1}{nm} \sum_{i=1}^n (n - i) \\ &= 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i \right) = 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2} \right) = 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} + \frac{\alpha}{2n} \end{aligned}$$

(из линейности математического ожидания). Таким образом, полное время, необходимое для проведения успешного поиска (включая время вычисления хеш-функции), составляет $\Theta(1 + \frac{\alpha}{2} + \frac{\alpha}{2n}) = \Theta(1 + \alpha)$. **О чем говорит проведенный анализ?** Если количество ячеек в хеш-таблице как минимум пропорционально количеству элементов, хранящихся в ней, то $\alpha = O(1)$

Плюсы и минусы

Главное преимущество использования цепочечных хеш-таблиц заключается в том, что они остаются эффективными, даже когда количество хранимых значений ключей намного превышает количество ячеек в хеш-таблице. Однако с увеличением количества хранимых ключей производительность цепочечных хеш-таблиц постепенно (линейно) ухудшается. Например, цепочечная хеш-таблица с 1000 ячеек памяти и 10 000 сохраненных ключей даст в 5–10 раз меньшую производительность по сравнению с цепочечными хеш-таблицами с 10 000 ячеек. Но цепочечная хеш-таблица все равно в 1000 раз быстрее простой хеш-таблицы. Другое преимущество использования цепочки для разрешения коллизий заключается в том, что ее производительность, в отличие от квадратичного зондирования, не ухудшается, когда таблица заполнена более чем наполовину. Этот метод абсолютно свободен от проблем кластеризации и, таким образом, обеспечивает эффективный механизм обработки коллизий.

Однако цепочечные хеш-таблицы наследуют недостатки связанных списков. Во-первых, для хранения значения ключа накладные расходы пространства следующего указателя в каждой записи могут быть значительными. Во-вторых, обход связанного списка имеет низкую производительность кэша, что делает кэш процессора неэффективным.



Хеширование бакетов

В закрытом хешировании все записи напрямую хранятся в хеш-таблице. Каждая запись со значением ключа k хранится в месте, называемом ее домашней позицией. Домашняя позиция вычисляется путем применения некоторой хеш-функции. В случае, если домашняя позиция записи с ключом k уже занята другой записью, то запись будет сохранена в каком-то другом месте в хеш-таблице. Это другое место будет определяться методом, который используется для разрешения коллизий. После вставки записей тот же алгоритм снова применяется для поиска определенной записи.

Одна из реализаций закрытого хеширования группирует хеш-таблицу в блоки, где M слотов хеш-таблицы делятся на B блоков. Таким образом, каждый блок содержит M/B слотов. Теперь, когда необходимо вставить новую запись, хэш-функция вычисляет исходную позицию. Если слот свободен, запись вставляется. В противном случае слоты бакеты последовательно просматриваются, пока не будет найден открытый слот. В случае, если весь бакет заполнен, запись вставляется в переполненный бакет. Переполненный бакет имеет бесконечную емкость в конце таблицы и является общим для всех. Эффективной реализацией хэширования ведра будет использование хэш-функции, которая равномерно распределяет записи между бакетами, так что в переполненный бакет нужно вставлять очень мало записей.

Другие хеш-функции

Одна из простых хеш-функций. Разработана Гленом Фаулером, Лондоном Керт Нолом и Фогном Во для общего применения.

```
unsigned int FNV1AHash(char *input)
{
    const unsigned int FNV_prime = 0x1000193;
    const unsigned int FNV_offset_basic = 0x811C9DC5;
    unsigned int hash = FNV_offset_basic;
    for (int i = 0; i < sizeof(input); ++i)
    {
        char byte_of_data = i;
        hash ^= byte_of_data;
        hash *= FNV_prime;
    }
    return hash;
}
```



Другие хеш-функции

Одна из простых хеш-функций. Разработана Бобом Дженкинсом для общего применения.

```
unsigned JOAATHash(char *input)
{
    unsigned int hash = 0;
    for (int i = 0; i < sizeof(input); ++i)
    {
        unsigned char byte_of_data = (unsigned char)i;
        hash += byte_of_data;
        hash += hash << 10;
        hash ^= hash >> 6;
    }
    hash += hash << 3;
    hash ^= hash >> 11;
    hash += hash << 15;
    return hash;
}
```



Другие хеш-функции

Одна из простых хеш-функций. Разработана Питером Вэйнбергером для общего применения.

```
unsigned int PJW32Hash(char *input)
{
    unsigned int hash = 0;
    for (int i = 0; i < sizeof(input); ++i)
    {
        unsigned char byte_of_data = (unsigned char)i;
        hash = (hash << 4) + byte_of_data;
        unsigned int h1 = hash & 0xf0000000;
        if (h1 != 0)
        {
            hash = ((hash ^ (h1 >> 24)) & (0xffffffff));
        }
    }
    return hash;
}
```



Универсальное хеширование

Если недоброжелатель будет умышленно выбирать ключи для хеширования с использованием конкретной хеш-функции, то он сможет подобрать n значений, которые будут хешироваться в одну и ту же ячейку таблицы, приводя к среднему времени выборки $\Theta(n)$. Решение - **универсальное хеширование**.

Главная идея универсального хеширования состоит в случайном выборе хеш-функции из некоторого тщательно отобранного класса функций в начале работы программы. Как и в случае быстрой сортировки, рандомизация гарантирует, что одни и те же входные данные не могут постоянно давать наихудшее поведение алгоритма. В силу рандомизации алгоритм будет работать всякий раз по-разному, даже для одних и тех же входных данных, что гарантирует высокую среднюю производительность для любых входных данных.

Пусть $\mathcal{H}$ - конечное множество хеш-функций, которые отображают данную совокупность ключей U в диапазон $\{0, 1, \dots, m - 1\}$. Такое множество называется универсальным, если для каждой пары различных ключей $k, l \in U$ количество хеш-функций $h \in \mathcal{H}$, для которых $h(k) = h(l)$, не превышает $|\mathcal{H}|/m$. Другими словами, при случайном выборе хеш-функции из $\mathcal{H}$ вероятность коллизии между различными ключами k и l не превышает вероятности совпадения двух случайным образом выбранных хеш-значений из множества $\{0, 1, \dots, m - 1\}$, которая равна $1/m$.



Универсальное хеширование

Напомним, $f: A \rightarrow B$, $f \in B^A$ (количество функций – размер B в степени размера A)

Тогда пусть $h \in \{0, \dots, m-1\}^k$, сделаем хеш-таблицу из m символов

l – случайное число. $|A| = n$. Найти сколько элементов в l ?

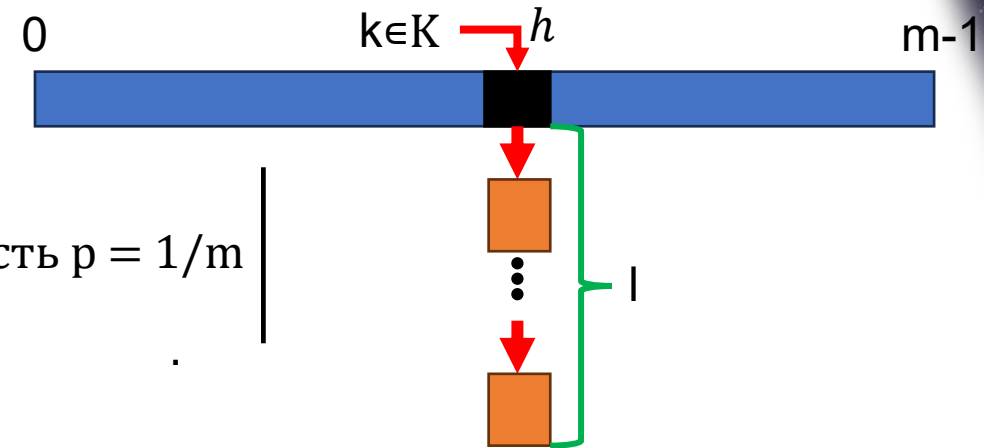
Теорема 12.3 Пусть хеш-функция h , случайным образом выбранная из универсального множества хеш-функций. Математическое ожидание $E[n_{h(k)}]$ длины списка, в котором находится ключ k , не превышает $1 + \alpha$.

$$E_h l = E_h \left\{ \sum_{a \in A} 1[h(a) = h(k)] \right\} = \sum_{a \in A} P_h[h(a) = h(k)]$$

2 случая:

$$= \begin{cases} 1. a \neq k. \text{ Очевидно выбор случаен, поэтому вероятность } p = 1/m \\ 2. a = k. p = 1 \end{cases}$$

$$\leq \frac{n}{m} + 1$$



В жизни все не так рандомно... h не может быть случайным. Множество функций слишком большое для памяти. Чтобы записать все в памяти, нужно $k \cdot \log(m) \sim \text{google} : -)$

Сделаем предположение послабее: $h \in \mathcal{H} \subseteq \{0, \dots, m-1\}^k$

$$\forall k' \neq k'' \in K P_h[h(k') = h(k'')] \leq \frac{1}{m}$$

Узнаем такие семейства..

Построение универсального класса

Сделаем предположение послабее: $h \in \mathcal{H} \subseteq \{0, \dots, m-1\}^k$

$$\forall k' \neq k'' \in K P_h[h(k') = h(k'')] \leq \frac{1}{m}$$

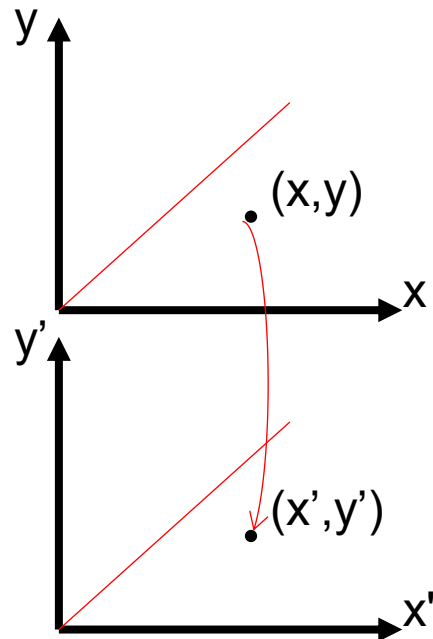
1 семейство $k = \{0, \dots, p-1\}$

$$h(x) = (ax + b) \bmod p \bmod m, \quad a \in \{1, \dots, p-1\} \in \mathbb{Z}_p^*, b \in \{0, \dots, p-1\} \in \mathbb{Z}_p, |\Omega| = p(p-1)$$

$\bmod m$ – это нужно для того чтобы уместится в тип, чисто арифметика без смысла

Оценим долю совпадений $P_h[h(x) = h(y)]$, $x, y \in \{0, \dots, p-1\} \in \mathbb{Z}_p$

$$P_{\substack{a \in \mathbb{Z}_p^* \\ b \in \mathbb{Z}_p}}[(ax + b) \bmod p \bmod m = (ay + b) \bmod p \bmod m] = P_{\substack{x', y' \in \mathbb{Z}_p \\ x' \neq y'}}[(x' \bmod m = y' \bmod m)]$$



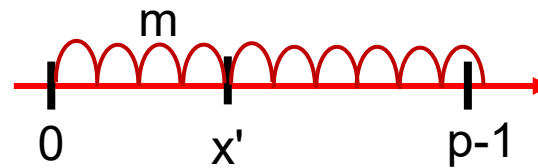
$$ax + b = ay + b \rightarrow x = y$$

$$x' = (ax + b) \bmod p$$

$$y' = (ay + b) \bmod p$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x & 1 \\ y & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} \bmod p$$

$$\det \neq 0$$



$$x' = y'$$

Перебираем x'

$$P_{\substack{x', y' \in \mathbb{Z}_p \\ x' \neq y'}}[(x' \bmod m = y' \bmod m)] \leq p \frac{\left\lfloor \frac{p}{m} \right\rfloor - 1}{p(p-1)} \leq \frac{\left\lfloor \frac{p}{m} \right\rfloor - 1}{(p-1)} = \frac{\frac{p-1}{m} - 1}{(p-1)} = \frac{1}{m}$$

Идеальное хеширование

$A \subseteq K, h: K \rightarrow \{0, \dots, m-1\}$

$\forall a' \neq a'' \in A \quad h(a') \neq h(a'')$ - нет коллизий

FKS hashing

Давайте случайно ткнуть $h \in \mathcal{H}$, сколько нужно раз ткнуть?

Теорема 12.5

Предположим, что n ключей сохраняются в хеш-таблице размером $m = n^2$ с использованием хеш-функции h , случайно выбранной из универсального класса хеш-функций. Тогда вероятность возникновения коллизий оказывается меньше $1/2$.

$P[h \text{ не имеет коллизий в } A] = P[\xi=0]$, ξ – коллизии

$\xi(a_i, a_j): i < j, h(a_i) = h(a_j)$

$$\xi = \sum_{a \in A} 1[h(a_i) = h(a_j)]$$

$$E\xi = \sum_{i < j} P_h[h(a_i) = h(a_j)] \leq \sum_{i < j} \frac{1}{m} = \frac{n(n-1)}{m}$$

$$P[h \text{ не имеет коллизий в } A] = 1 - P[\xi \geq 1] \geq 1 - \frac{E\xi}{1} \geq 1 - \frac{n(n-1)}{m}$$

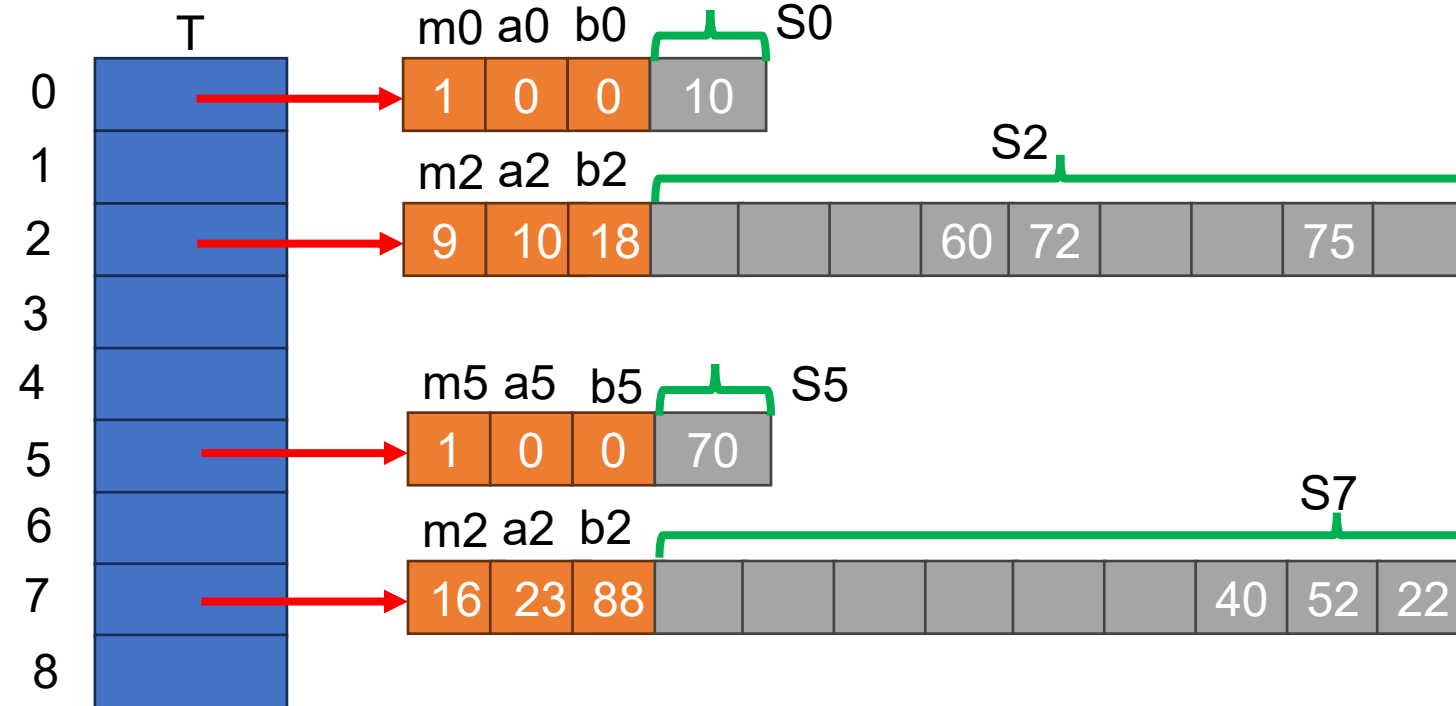
Не хочется иметь таблицу размера $n(n-1)$

Попробуем линейность делая таблицу в таблице.

(Применение неравенства Маркова, $\Pr\{X \geq t\} \leq E[X]/t$) - дз

Идеальное хеширование

Для создания схемы идеального хеширования мы используем двухуровневую схему хеширования с универсальным хешированием на каждом уровне.



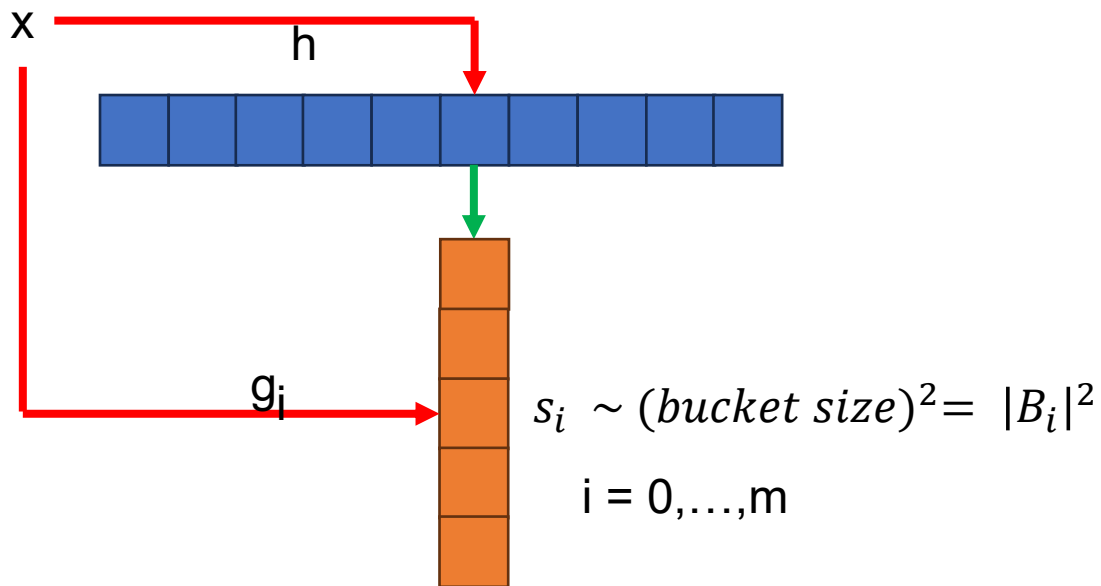
Использование идеального хеширования для хранения множества $K = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}$.

Внешняя хеш-функция имеет вид $h(k) = ((ak + b) \bmod p) \bmod m$, где $a = 3$, $b = 42$, $p = 101$ и $m = 9$.

Чтобы гарантировать отсутствие коллизий на втором уровне, требуется, чтобы размер m , хеш-таблицы S , был равен квадрату числа n , ключей, хешированных в ячейку j . Ожидаемое количество требуемой для хеш-таблицы $O(n)$.

Хеш-функция первого уровня выбирается из класса $\mathcal{H}_{p,m}$, p является простым числом, превышающим значение любого из ключей. Ключи, хешированные в ячейку j , затем повторно хешируются во вторичную хеш-таблицу S_j , размером m_j , с использованием хеш-функции h_j , выбранной из класса $\mathcal{H}_{p,m_j}^2$.

Идеальное хеширование



Теорема 12.6

Предположим, что мы сохраняем n ключей в хеш-таблице размером $m = n$ с использованием хеш-функции h , выбираемой случайным образом из универсального класса хеш-функций. Тогда мы имеем

$$E \left[\sum_{j=0}^{m-1} n_j^2 \right] < 2n,$$

где n_j - количество ключей, хешированных в ячейку j .

Количество
коллизий

$$\sum_i s_i = \sum_i |B_i|^2 = \sum_i 2 \frac{|B_i|(|B_i| - 1)}{2} + |B_i| = 2 \sum_i \binom{|B_i|}{2} + \sum_i |B_i| = 2\xi + n$$

$$E \left[\sum_i s_i \right] = 2E\xi + n \leq 2 \frac{n(n-1)}{2n} + n \leq 2n$$

Доказательство. Мы имеем

Идеальное хеширование

Следствие 12.2 Если мы сохраняем n ключей в хеш-таблице размером $m = n$ с использованием хеш-функции h , выбираемой случайным образом из универсального класса хеш-функций, и устанавливаем размер каждой вторичной хеш-таблицы равным $m_j = n_j^2$, $j = 0, 1, \dots, m - 1$, то математическое ожидание количества необходимой для вторичных хеш-таблиц в схеме идеального хеширования памяти не превышает величины $2n$.

Доказательство. Поскольку $m_j = n_j^2$ для $j = 0, 1, \dots, m - 1$, теорема 12.6 дает

$$E \left[\sum_{j=0}^{m-1} m_j \right] = E \left[\sum_{j=0}^{m-1} n_j^2 \right] < 2n$$

Следствие 12.3 Если мы сохраняем n ключей в хеш-таблице размером $m = n$ с использованием хеш-функции h , выбираемой случайным образом из универсального класса хеш-функций, и устанавливаем размер каждой вторичной хеш-таблицы равным $m_j = n_j^2$, $j = 0, 1, \dots, m - 1$, то вероятность того, что общее количество необходимой для вторичных хеш-таблиц памяти не менее $4n$, меньше, чем $1/2$.

Доказательство. Вновь применим неравенство Маркова, $\Pr\{X \geq t\} \leq E[X] / t$, на этот раз - к неравенству, с $X = \sum_{j=0}^{m-1} m_j$ и $t = 4n$:

$$P \left[\sum_i s_i \leq 4n \right] \geq 1 - \frac{2n}{4n} = \frac{1}{2}$$

Плюсы и минусы хеширования

- ✓ Для хранения индекса не требуется дополнительного места, как в случае с другими структурами данных.
- ✓ Хеш-таблица обеспечивает быстрый доступ к данным и дополнительное преимущество быстрых обновлений.
- ✗ Для вставки и извлечения значений ей обычно не хватает локальности и последовательного извлечения по ключу. Это делает вставку и извлечение значений данных еще более случайными. Тем более, выбор эффективной хеш-функции — это скорее искусство, чем наука. Нередко (в хеш-таблицах с открытой адресацией) создается плохая хеш-функция.



Реальные приложения хеширования

Базы данных компакт-дисков

Для компакт-дисков желательно иметь всемирную базу данных компакт-дисков, чтобы, когда пользователи вставляют свой диск в проигрыватель компакт-дисков, они получали полную таблицу содержания на экране своего компьютера.

Водительские права/страховые карты

Как и в нашем примере с компакт-диском, даже номера водительских прав или страховых карт создаются с помощью хеширования из элементов данных, которые никогда не меняются: дата рождения, имя и т. д.

Разреженная матрица

Разреженная матрица — это двумерный массив, в котором большинство записей содержат 0. То есть в разреженном массиве очень мало ненулевых записей. Используя хеширование, мы можем хранить двумерный массив в одномерном массиве.

Подписи файлов

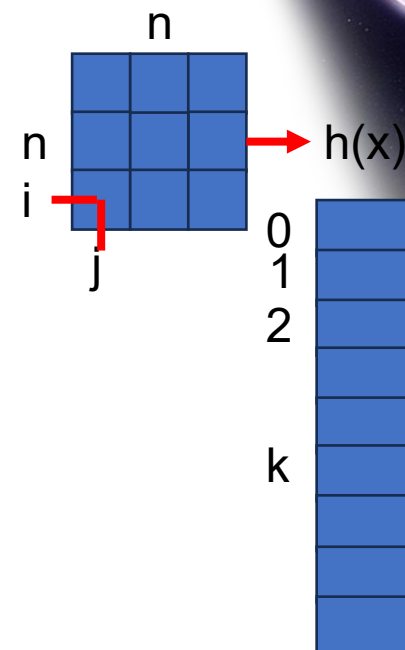
Подписи файлов предоставляют компактное средство идентификации файлов.

Игровые доски

На игровой доске для крестиков-ноликов или шахмат, положение в игре может храниться с помощью хэш-функции.

Графика

В графике центральной проблемой является хранение объектов в сцене или представлении.



Agenda

**Хеш-функции и
хеш-таблицы**

85 минут

**Указатель на
функцию**

5 минут

Причудливые объявления

Язык C позволяет создавать сложные формы данных. Хотя мы придерживаемся простейших форм, все же имеет смысл отметить некоторые из доступных возможностей. Когда вы делаете объявление, имя (или идентификатор) можно изменить, добавив модификатор.

Модификатор	Описание
*	Обозначает указатель
()	Обозначает функцию
[]	Обозначает массив

```
int board[8][8]; // массив из массивов значений int
```

```
int ** ptr; // указатель на указатель на int
```

```
int * risks[10]; // 10-элементный массив указателей на int
```

```
int (* rusks ) [10]; // указатель на массив из 10 значений int
```

```
int * oof[3][4]; // массив размером 3 x 4 указателей на int
```

```
int (*uuf)[3][4]; // указатель на массив размером 3 x 4 значений int
```

```
int (*uof [3])[4]; // 3-элементный массив указателей на 4-элементные массивы значений int
```

```
char * fump(int); // функция, возвращающая указатель на char
```

```
char (* frump)(int); // указатель на функцию, возвращающую тип char
```

```
char (* flump[3])(int); // массив из 3 указателей на функции, которые возвращают тип char
```


Функции и указатели

Как продемонстрировало обсуждение объявлений, допускается объявлять указатели на функции. Возможно, вас интересует, в чем они могут быть полезны.

Обычно указатель на функцию используется в качестве аргумента в другой функции, сообщая ей, какую функцию применять. Например, сортировка массива предполагает сравнение двух элементов для выяснения того, какой из них должен следовать первым. В случае числовых элементов можно использовать операцию $>$. Но в целом элементом может быть строка или структура, что требует вызова специальной функции для выполнения сравнения. Функция `qsort()` из библиотеки `C` спроектирована на работу с массивами любого вида при условии, что вы уведомите ее о том, какую функцию применять для сравнения элементов. С этой целью `qsort()` принимает в одном из своих аргументов указатель на функцию. Затем `qsort()` использует указанную функцию для сортировки значений определенного типа — будь он целочисленным, строкой или структурой.



Функции и указатели

`void ToUpper(char *);` // преобразует строку в верхний регистр

Тип `ToUpper()` определен как “функция с параметром `char *` и возвращаемым типом `void`”. Вот как объявить указатель на функцию такого типа по имени `pf`:

`void (*pf)(char *);` // `pf` - указатель на функцию

Читая это объявление, вы видите, что первая пара круглых скобок связывает операцию `*` с `pf`, т.е. `pf` является указателем на функцию. Это делает `(*pf)` функцией, а `(char *)` — списком ее параметров функции и `void` — возвращаемым типом.

`void *pf(char *);` // `pf` - функция, которая возвращает указатель

В этом контексте для представления адреса функции может применяться ее имя:

`void ToUpper(char *)`

`void ToLower(char *)`

`int round(double);`

`void (*pf) (char *);`

`pf = ToUpper;` // допустимо, `ToUpper` - адрес функции

`pf = ToLower;` // допустимо, `ToLower` - адрес функции

`pf = round;` // недопустимо, `round` - неподходящий тип функции

`pf = ToLower();` // недопустимо, `ToLower()` не является адресом



Функции и указатели

Подобно тому, как можно применять указатель на данные с целью доступа к ним, вы можете использовать указатель на функцию для обращения к этой функции. На удивление для этого существуют два логически несогласованных синтаксических правила, как иллюстрируется в следующем фрагменте:

```
void ToUpper(char *);  
void ToLower(char *);  
void (*pf) (char *);  
char mis [] = "Nina Metier";  
pf = ToUpper;  
(*pf)(mis); // применить ToUpper к mis (первый синтаксис)  
pf = ToLower;  
pf(mis); // применить ToLower к mis (второй синтаксис)
```

Компилятор K & R C не разрешает вторую форму, но для поддержки совместимости с существующим кодом стандарт ANSI C принимает обе формы ((*pf) (mis) и pf(mis)) как эквивалентные. Последующие стандарты сохранили такой в высшей степени двойственный подход.



Функции и указатели

Одним из наиболее распространенных случаев использования указателей на данные является аргумент функции, и то же самое относится к указателю на функцию.

Например, рассмотрим следующий прототип функции:

```
void show(void (* fp) (char *), char * str);
```

Он выглядит запутанным, но в нем объявляются два параметра, `fp` и `str`. Параметр `fp` — это указатель на функцию, а `str` — указатель на данные. Точнее, `fp` указывает на функцию, которая принимает параметр `char *` и имеет возвращаемый тип `void`, а `str` указывает на `char`. Таким образом, имея представленные выше объявления, можно делать вызовы функций вроде приведенных ниже:

```
show(ToLower, mis); /* show () использует функцию ToLower(): fp = ToLower * /  
show(pf, mis) ; /*show() использует функцию, указанную посредством pf: fp = pf */
```

Каким образом `show()` применяет переданный указатель на функцию?

Для вызова этой функции в `show()` используется либо синтаксис `fp()`, либо синтаксис `(*fp) ()`:

```
void show(void (* fp)(char *), char * str)  
{  
    (*fp)(str); /* применить выбранную функцию к str */  
    puts(str); /* отобразить результат */  
}
```

Здесь функция `show ()` сначала трансформирует строку `str`, применяя к ней функцию, на которую указывает `fp`, после чего отображает результирующую строку.